

Entwicklungs-Dokumentation

Digital & Mobil in Deutschland

Von der ersten HTML-Datei zum mehrsprachigen Informationsportal

Dieses Dokument beschreibt die schrittweise Entwicklung des Projekts — jede Phase enthält Code-Beispiele aus dem Projekt.

DEUTSCHE VERSION

Phase 1 — Grundstruktur und Glasmorphismus-Design

Das Projekt begann mit einfachen index.html und style.css Dateien. Als visuelle Ausrichtung wurde Glasmorphismus gewählt — halbtransparente Elemente mit Weichzeichner-Effekt (Blur) über einem Hintergrundbild.

CSS-Variablen und Hintergrund (css/base.css)

```
:root {
  --glass-bg: rgba(255, 255, 255, 0.6);
  --glass-border: rgba(255, 255, 255, 0.3);
  --primary-blue: #00529b;
  --text-dark: #1d1d1f;
}

body::before {
  content: "";
  position: fixed;
  inset: 0; z-index: -1;
  background: linear-gradient(rgba(255,255,255,0.1), rgba(255,255,255,0.1)),
    url('../img/...jpeg') no-repeat center center;
  background-size: cover;
}
```

Karten im Glasmorphismus-Stil (css/components.css)

```
.card {
  background: rgba(255, 255, 255, 0.2) !important;
  backdrop-filter: blur(15px);
  -webkit-backdrop-filter: blur(15px);
  border: 1px solid var(--glass-border);
  border-radius: 28px;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.1);
  animation: fadeInUp 0.8s ease-out forwards;
}
```

Lokale Schriftarten statt Google Fonts CDN (css/base.css)

Geladen von <https://gwfh.mranftl.com/fonts/montserrat?subsets=latin> und im Ordner fonts/ abgelegt

```
@font-face {
  font-family: 'Montserrat';
  font-weight: 400;
  font-display: swap;
  src: url('../fonts/montserrat-v31-latin-regular.woff2') format('woff2');
}

/* + Schriftschnitte 600, 700, 800 */
```

font-display: swap — Text wird sofort mit Fallback-Schrift angezeigt. DSGVO-relevant: Keine externen Anfragen an Google-Server.

Phase 2 — Wetter-Widget (js/weather.js)

Das Wetter-Widget ist über die OpenWeatherMap API angebunden und als horizontales Scroll-Panel im Header umgesetzt.

API-Anfrage über fetch() mit Fehlerbehandlung

```
const WEATHER_API_KEY = '...';
let currentCity = 'Hattingen';

async function getWeather() {
  try {
    const res = await fetch(
      `https://api.openweathermap.org/data/2.5/weather
      ?q=${currentCity}&appid=${WEATHER_API_KEY}&units=metric&lang=de`
    );
    const d = await res.json();

    if (!d.main) {
      alert('Stadt nicht gefunden!');
      return;
    }
  } catch (e) {
    console.error('Wetter-Fehler:', e);
  }
}
```

Wetter-Icon nach Bedingungscode

```
const code = d.weather[0].id;
let icon = '☁';
if (code === 800) icon = '☀';
else if (code > 800) icon = '☁';
else if (code >= 600) icon = '❄';
else if (code >= 300) icon = '☁';
```

Änderung der Stadt per Klick

Ursprünglich wurde GPS über navigator.geolocation abgefragt. Jetzt ist Hattingen Standard — der Benutzer kann die Stadt per Klick ändern. API gibt den Namen immer auf Deutsch zurück (lang=de):

```
tempEl.style.cursor = 'pointer';
tempEl.onclick = () => {
  const newCity = prompt('Stadt eingeben:', currentCity);
  if (newCity && newCity.trim() !== '') {
    currentCity = newCity.trim();
    getWeather();
  }
};
// Druckumrechnung
pressEl.innerText = Math.round(d.main.pressure * 0.75006); // hPa → mmHg
```

Phase 3 — Echtzeit-Uhr und Datum (js/clock.js)

Uhrzeit und Datum werden im deutschen Format angezeigt und im Sekundentakt aktualisiert:

```
function updateClock() {
  const now = new Date();
  const timeStr = now.toLocaleTimeString('de-DE', {
    hour: '2-digit', minute: '2-digit', second: '2-digit'
  });
  document.getElementById('current-time').innerText = timeStr;
}

function initClock() {
  const dateEl = document.getElementById('current-date');
  dateEl.innerText = new Date().toLocaleDateString('de-DE',
    { day: 'numeric', month: 'long' }).replace('.', '');
  updateClock();
  setInterval(updateClock, 1000);
}
```

Top-Bar: Uhr + Wetter als eine Einheit (css/topbar.css)

```
.top-bar {
  background: rgba(255, 255, 255, 0.15);
  backdrop-filter: blur(10px);
  position: sticky; top: 0; z-index: 1100;
}

.w-item {
  background: rgba(255, 255, 255, 0.2);
  border: 1px solid var(--glass-border);
  border-radius: 10px;
}

.w-item:active { transform: scale(0.95); }

.divider { color: rgba(0, 0, 0, 0.2); }
```

Tooltip und Scroll-Button

```
<!-- Tooltip beim Klick -->
<span class="w-item" onclick="toggleLabel(this)">
  ◊ <span class="w-label">Luftfeuchtigkeit</span> <span id="hum">--</span>%
</span>

<!-- Scroll-Button für das Wetter-Panel -->
<button class="weather-arrow-btn" onclick="toggleWeatherScroll()">↻</button>
```

Phase 4 — Tab-Navigation (js/tabs.js, css/navbar.css)

Eine SPA-Navigation wurde entwickelt — der Inhalt wechselt ohne Neuladen der Seite.

Tab-Wechsel mit Neustart der Animation

```
function showTab(tabId, event) {
  document.querySelectorAll('.tab-content')
    .forEach(t => t.classList.remove('active'));
  document.querySelectorAll('.nav-btn')
    .forEach(b => b.classList.remove('active'));

  const targetTab = document.getElementById(tabId);
  if (targetTab) {
    targetTab.style.animation = 'none';
    void targetTab.offsetHeight; // Erzwingener Reflow
    targetTab.style.animation = '';
    targetTab.classList.add('active');
  }
  window.scrollTo(0, 0);
}
```

void targetTab.offsetHeight — ein Trick, der den Browser zwingt, die Animation von vorn zu starten.

Navbar mit Glasmorphismus (css/navbar.css)

```
.navbar {
  background: rgba(255, 255, 255, 0.3);
  backdrop-filter: blur(20px);
  position: sticky; top: 45px; z-index: 1000;
}
.nav-btn.active {
  background: var(--primary-blue);
  color: white;
  box-shadow: 0 4px 12px rgba(0, 82, 155, 0.2);
}
```

Scroll-Pfeile, Swipe-Hint und „Nach oben“-Button

```
// Scroll-Pfeile
function scrollTabs(direction) {
  const viewport = document.querySelector('.nav-scroll-viewport');
  viewport.scrollTo({
    left: direction === 1 ? viewport.scrollWidth : 0,
    behavior: 'smooth'
  });
}

// Swipe-Hint verschwindet beim ersten Scrollen
let swipeHintDone = false;
function hideSwipeHint() {
  if (!swipeHintDone) {
    swipeHintDone = true;
    document.querySelector('.scroll-hint-left')?.classList.add('hidden');
  }
}
```

```
// Nach-oben-Button: zwei Zustände
// Während Scrollen: klein + halbtransparent
// Nach 150ms Stillstand: groß + sichtbar
scrollTopBtn?.addEventListener('pointerdown', (e) => {
  e.preventDefault();
  window.scrollTo({ top: 0, behavior: 'smooth' });
});
```

Phase 5 — Refactoring und Modularisierung

Mit dem Wachstum des Projekts wurde der Code in Module aufgeteilt:

Vorher: style.css + script.js (jeweils eine Datei)

Nachher:

CSS-Modul	Zuständigkeit
css/base.css	Variablen, Reset, Hintergrund, Schriftarten, Animationen
css/topbar.css	Sticky-Header mit Uhr und Wetter
css/navbar.css	Tab-Navigation mit Glasmorphismus
css/components.css	Karten, Buttons, Tabs, iFrames, News-Karten, Consent-Placeholder

JS-Modul	Zuständigkeit
js/main.js	Initialisierung, Hash-Routing, YouTube, Cookie-Icon
js/tabs.js	Tab-Wechsel, Scroll-Pfeile, Swipe-Hint, Scroll-to-Top
js/weather.js	OpenWeatherMap API, Stadtänderung per Klick
js/clock.js	Uhrzeit und Datum in Echtzeit auf Deutsch
js/news.js	RSS-Feed (tagesschau.de / Euronews), je nach Sprache
js/components.js	Web Components: <app-header>, <scroll-hint>
js/cookie-consent.js	Silktide-Konfiguration mit onAccept/onReject-Logik

Phase 6 — RSS-Nachrichten (js/news.js)

Die Nachrichten werden je nach Sprache geladen — für die deutsche Version von tagesschau.de, für die russische von Euronews:

```
const feeds = {
  de: 'https://www.tagesschau.de/...rss2.xml',
  ru: 'https://ru.euronews.com/rss'
};

data.items.slice(0, 5).forEach(item => {
  div.innerHTML = `
    <a href="${item.link}" class="news-title">${item.title}</a>
    <p class="news-desc">${item.description.split('.')[0]}...</p>
  `;
});
```

Die Quellenangabe wird dynamisch aktualisiert: „Quelle: tagesschau.de“ oder „Источник: ru.euronews.com“.

Phase 7 — Eigene Backend-Anwendungen (iFrame)

Zwei eigenständige Tools wurden als separate Projekte entwickelt und über iFrames eingebunden. Laden nur nach Zustimmung (Consent):

```
function loadPdfCompressor() {
    iframe.src = 'https://pdf-compressor-web.onrender.com';
    iframe.style.display = 'block';
    ph.style.display = 'none';
}

// Hintergrund-Ping zum Aufwecken von Render
fetch('https://...onrender.com/wakeup', { mode: 'no-cors' }).catch(() => {});
```

Consent-Placeholder in HTML: Gestrichelter Rahmen mit Button — erst nach Klick oder Cookie-Zustimmung wird das iFrame geladen.

Phase 8 — YouTube-Integration und Datenschutz

YouTube-Videos werden im erweiterten Datenschutzmodus geladen — nur nach einer Benutzeraktion (Zwei-Klick-Lösung). Autoplay wurde entfernt (Android-Problem):

```
function loadVideo(placeholderId, iframeId, videoId) {
    iframe.src = `https://www.youtube-nocookie.com/embed/${videoId}?rel=0`;
    iframe.style.display = 'block';
    placeholder.style.display = 'none';
}

// Videoformate
css: .video-wrapper.horizontal { aspect-ratio: 16 / 9; }
css: .video-wrapper.vertical { aspect-ratio: 9 / 16; max-width: 350px; }
```

Phase 9 — Cookie Consent (js/cookie-consent.js)

Der Silktide Consent Manager wurde als ZIP-Archiv von der offiziellen Website heruntergeladen und gemäß der beigefügten README installiert. Drei Cookie-Kategorien:

Kategorie	ID	Steuert
Notwendig (Pflicht)	necessary	Grundfunktionen der Website
Statistik	analytics	RSS-Nachrichten (tagesschau / Euronews)
Werbung & externe Inhalte	advertising	YouTube, PDF-Kompressor, Foto-zu-PDF

```
// onAccept('analytics')
loadNews(); // Nachrichten laden

// onAccept('advertising')
// Alle YouTube-Placeholder automatisch aktivieren
document.querySelectorAll('[id^="video-placeholder-"]')
    .forEach(ph => ph.querySelector('button')?.click());

// Cookie-Icon beim Scrollen ausblenden
if (window.scrollY > 0) {
    icon.style.setProperty('opacity', '0', 'important');
}
```

Phase 10 — Mehrsprachigkeit und Web Components

Web Component: <app-header>

Der gesamte Header ist als Custom Element umgesetzt — einmal definiert, auf allen Seiten einheitlich:

```
class AppHeader extends HTMLElement {
  connectedCallback() {
    const isRu = document.documentElement.lang === 'ru';
    const texts = {
      loading: isRu ? 'Загрузка...' : 'Laden...',
      humidity: isRu ? 'Влажность' : 'Luftfeuchtigkeit',
      city: isRu ? 'Город Хаттинген' : 'Stadt Hattingen'
    };
  }
};
```

Navigation zwischen Sprachen und Ordern

```
const path = window.location.pathname;
const filename = path.split('/').pop() || 'index.html';

if (isRu) {
  // ru/magnet.html → /tabs/magnet.html
  langHref = filename === 'index.html' ? '/' : '/tabs/' + filename;
} else {
  // /tabs/magnet.html → /ru/magnet.html
  langHref = path === '/' ? '/ru/' : '/ru/' + filename;
}
```

Verwendung in HTML

```
<app-header></app-header>
<scroll-hint></scroll-hint>
```

Hash-Routing für Deep-Linking (js/main.js)

```
const hash = window.location.hash.replace('#', '');
if (hash) {
  showTab(hash, { currentTarget: targetButton, ... });
  history.replaceState(null, null, window.location.pathname);
}
```

Phase 11 — Testphase der produktiven Website

Bericht: Behebung des 404-Fehlers beim Klick auf E-Mail

Problembeschreibung:

Bei der Testphase trat auf Android-Geräten beim Klick auf „E-Mail schreiben“ der Serverfehler 404 NOT_FOUND auf.

Ursache:

Der interne JavaScript-Router hat den Klick auf den mailto:-Link abgefangen. Statt den E-Mail-Client zu öffnen, hat der Android-Browser den Link als interne Seite behandelt und eine 404-Fehlerseite zurückgegeben. Auf iOS trat der Fehler nicht auf, da Safari mailto:-Links auf Betriebssystemebene erzwingt.

Durchgeführte Änderungen:

Dem <a>-Tag wurden target="_blank" und rel="noopener noreferrer" hinzugefügt:

```
// Vorher:
<a href="mailto:hattingen325@gmail.com" style="...">

// Nachher:
<a href="mailto:hattingen325@gmail.com"
  target="_blank" rel="noopener noreferrer" style="...">
```

Ergebnis:

Das Problem wurde vollständig behoben. Der mailto:-Link wird auf Android nun korrekt an das Betriebssystem weitergegeben und öffnet den E-Mail-Client.

Ergebnis

Von einer einzigen HTML-Datei zu einer modularen, mehrsprachigen SPA mit:

- 7 eigenständigen JavaScript-Modulen + Cookie Consent Konfiguration
- 4 CSS-Modulen mit einheitlichem Glasmorphismus-Design
- 2 Web Components (<app-header>, <scroll-hint>) für eine konsistente UI
- 2 eigenen Backend-Anwendungen (Python/Flask, Python/Streamlit)
- API-Integration: OpenWeatherMap, rss2json
- DSGVO-Konformität: Cookie Consent, Zwei-Klick-Lösung, lokale Schriftarten, Impressum, Datenschutz

Mobile-only Entwicklung: Der gesamte Code wurde auf dem iPhone geschrieben und committed. Die hohe Frequenz der Commits ist auf das Testen von Mikro-Änderungen direkt über den Deploy-Prozess zurückzuführen.

Фаза 1 — Базовая структура и дизайн в стиле стекломорфизма

Проект начался с простых файлов `index.html` и `style.css`. Выбран стекломорфизм — полупрозрачные элементы с эффектом размытия поверх фонового изображения. CSS-переменные (`--glass-bg`, `--glass-border`, `--primary-blue`, `--text-dark`) определяют всю цветовую схему. Шрифты `Montserrat` размещены локально (DSGVO: нет запросов к Google).

Фаза 2 — Виджет погоды (`js/weather.js`)

Подключён через `OpenWeatherMap API`. Хаттинген по умолчанию вместо GPS. Город можно изменить по клику — даже при вводе на русском API показывает немецкое название. Обработка ошибок: скрипт не ломается при неверном вводе. Давление конвертируется из гПа в мм рт.ст. ($\text{pressure} * 0.75006$).

Фаза 3 — Часы и дата (`js/clock.js`)

Время обновляется каждую секунду, дата устанавливается однократно. Top-Bar со стекломорфизмом (`sticky`, `blur:10px`, `z-index:1100`). Элементы погоды с разделителями, `scale(0.95)` при нажатии, кнопка прокрутки ↕.

Фаза 4 — SPA-навигация (`js/tabs.js`)

SPA-навигация без перезагрузки. `showTab()` с `reflow`-трюком (`void offsetHeight`). Навбар со стекломорфизмом (`blur:20px`, `top:45px`). Стрелки прокрутки меню. `Swipe-Hint` с анимированным пальцем, исчезает при первом скролле. Кнопка «Наверх» с двумя состояниями (маленькая при скролле, большая через 150мс), `pointerdown` для тача.

Фаза 5 — Рефакторинг и модуляризация

Код разделён на модули: 4 CSS-файла (`base`, `topbar`, `navbar`, `components`) и 7 JS-файлов (`main`, `tabs`, `weather`, `clock`, `news`, `components`, `cookie-consent`).

Фаза 6 — RSS-новости (`js/news.js`)

Новости загружаются в зависимости от языка: `tagesschau.de` (DE) или `Euronews` (RU). Максимум 5 статей. Источник обновляется динамически. Функция экспортируется как `window.loadTagesschauNews`.

Фаза 7 — Собственные бэкенд-приложения (`iFrame`)

Фото в PDF (`Streamlit`) и PDF-компрессор (`Flask/Render`) встроены через `iFrame`. `src` пуст до согласия (DSGVO). Фоновый пинг будит `Render` заранее.

Фаза 8 — YouTube и защита данных

Двухклик-решение: placeholder → клик → youtube-nocookie.com. Autoplay удалён (проблема на Android). Форматы: 16:9 (.horizontal) и 9:16 (.vertical, макс. 350px).

Фаза 9 — Cookie Consent

Silktide Consent Manager скачан ZIP-архивом. Три категории: necessary (обязат.), analytics (RSS), advertising (YouTube, PDF). При onReject все iFrame-src очищаются. Иконка cookies скрывается при скролле.

Фаза 10 — Многоязычность и Web Components

<app-header> — Custom Element со словарём переводов. Автоматическая навигация /tabs/ ↔ /ru/ по имени файла. <scroll-hint> — подсказка свайпа. Оба компонента читают <html lang="...">. Hash-маршрутизация для прямых ссылок (index.html#jobs).

Фаза 11 — Тестирование продуктивного сайта

Проблема: на Android при клике на «E-Mail schreiben» возникала ошибка 404. Причина: JS-роутер перехватывал mailto:-ссылку как внутреннюю страницу. На iOS не воспроизводилось (Safari перехватывает mailto на уровне ОС). Решение: добавлены target="_blank" и rel="noopener noreferrer".

Результат

От одного HTML-файла до модульного, многоязычного SPA:

- 7 JS-модулей + Cookie Consent
- 4 CSS-модуля со стекломорфизмом
- 2 Web Components (<app-header>, <scroll-hint>)
- 2 бэкэнд-приложения (Python/Flask, Python/Streamlit)
- API: OpenWeatherMap, rss2json
- DSGVO: Cookie Consent, двухклик, локальные шрифты, Impressum, Datenschutz

Mobile-only разработка: весь код написан и закоммичен на iPhone. Высокая частота коммитов связана с тестированием микро-изменений напрямую через deploy-процесс.